

Procesamiento de archivos Log

Integrantes:

Chaves, Laura

Lopez, Melina

Saldaña, Guillermo

Consigna:

- 1) Identificar las variables de las que se pueden obtener métricas y datos cualitativos.
- 2) Elaborar código reutilizable para procesar al menos 3 de esas variables.
- 3) Diseñar un dashboard para representarlas

NOTA: Justificar todas las acciones tomadas con un marco teórico adecuado

Metodología:

Se implementó un pipeline de logs: desde R (`readr/stringr/httr/data.table`) se ingesta el archivo por *chunks* y se almacena crudo en ClickHouse (`MergeTree`). El parseo se realiza en ClickHouse con expresiones regulares y su resultado se materializa para consulta. Sobre esa capa se construyó un dashboard en Shiny (`ggplot2/plotly/treemapify`) con KPIs y visualizaciones.

¿Por qué se adapta a la lectura de cualquier tipo de archivo de log?

El proceso se adapta tanto al formato como al tamaño del log: primero detecta el tipo (`combined`, `JSONL` o `texto`) y, ante variantes, ingesta en crudo; además, la carga se realiza por *chunks* para manejar archivos grandes sin comprometer memoria ni estabilidad. El parseo se resuelve en el motor con expresiones regulares y el resultado se guarda en una vista materializada que se actualiza con cada inserción y acelera las consultas. Si cambian los patrones del log, se ajusta esa lógica y se regenera la vista a partir del crudo, sin reingestar el archivo.

Métricas seleccionadas y su justificación:

- **HTTP Status (tasa de error, familia de status):** se calcula la tasa de error (respuestas con código ≥ 400) y se agrupan los códigos por familia. Esta métrica es fundamental para evaluar la calidad del servicio y la observabilidad del sistema, ya que permite identificar rápidamente problemas de disponibilidad, errores de cliente o servidor y tendencias en el comportamiento de las respuestas.
- **URL / Endpoint (top endpoints, agrupación por primer segmento):** se identifican los endpoints más solicitados y se agrupan las URLs por su primer segmento. Esto facilita el análisis de patrones de uso y permite detectar puntos de acceso críticos (“hotspots”), ayudando a priorizar optimizaciones y entender cómo los usuarios interactúan con el sistema.
- **Método HTTP (GET, POST, etc.):** se analiza la frecuencia de cada método HTTP para visualizar la proporción de consultas (lecturas) y escrituras, así como posibles variaciones o anomalías en el tráfico. Esta información es clave para ajustar recursos, identificar cambios en el uso y anticipar necesidades de escalabilidad.
- **Bytes (promedio de bytes transferidos):** sirve como referencia rápida del costo de transferencia y del tamaño de las respuestas enviadas. Permite detectar posibles excesos en consumo de ancho de banda o identificar endpoints que generan cargas inusuales.
- **User-Agent (familias de navegador):** se agrupan los User-Agent en familias de navegadores (Chrome, Firefox, Safari, Edge, Opera, Bots y Otros) para segmentar el tipo de clientes que acceden al sistema, diferenciando entre usuarios humanos y bots, y observando el market share de cada navegador. Esta segmentación es útil para adaptar optimizaciones y detectar automatizaciones no deseadas.

Las métricas presentadas son agregaciones por columnas (count, avg, group by), propias de la analítica OLAP, donde las column stores superan a las de filas por su compresión y ejecución vectorizada.

Se utilizó escala logarítmica en endpoints y métodos debido a una distribución sesgada; así se pueden comparar órdenes de magnitud evitando que valores altos dominen la visualización.

¿Por qué elegimos ClickHouse? Y estrategia de carga

Elegimos ClickHouse debido a que es una base de datos orientada a columnas, pensada específicamente para la analítica a gran escala.

Su diseño está optimizado para realizar lecturas masivas y operaciones de agregación, como los group by, además de contar con compresión por columna y utilizar MergeTree como motor base para trabajar con grandes volúmenes de datos. ClickHouse demuestra un rendimiento sobresaliente en el manejo de patrones de logs, donde la carga suele ser append-only y se requieren consultas de agregación eficientes.

La estrategia de ingesta se implementó en dos etapas:

- 1- Carga cruda: se carga el archivo crudo en la tabla bi.access_log_auto, enviando los datos desde R en bloques de 100 mil líneas por vez, almacenando cada registro como una línea cruda junto a un timestamp opcional.

Esto se pensó para maximizar la robustez ante variantes y errores, separando el proceso de ingesta del de parseo, lo que permite reprocesar y evolucionar el parsing sin volver a leer el archivo original.

¿Por qué usamos Shiny?

Se encuentra en el mismo entorno (R) que el proceso de ingesta y permite un prototipado ágil. Además, presenta una integración eficiente con ggplot2 y plotly, ofreciendo capacidades de interactividad como el drill-down, lo que reduce la necesidad de emplear herramientas externas y disminuye la fricción operativa. En los contextos de demostración o entrega de prácticas, optimiza el coste de integración manteniendo un alto nivel de expresividad.